

Contents

Fault-Zone Projection-Quality Plan (v1)	1
1. Why the fault-zone region looks worst (precise diagnosis)	1
1.A — Fault-trace coincides with basin edges	1
1.B — Fault refinement creates a 2-D band of fine tets around a 1-D feature	2
1.C — H1-P1 forces continuity across the fault	2
2. Tools the codebase ALREADY provides	2
3. Design space	3
Lever T1 — Switch material to DG $L^2(0)$ (per-element constant)	3
Lever T2 — DG $L^2(p \geq 1)$: keep within-tet curvature, allow face jumps	3
Lever T3 — Per-side fault-coupled projection	4
Lever T4 — Volume averaging of the source field at projection time	4
Lever T5 — Sidecar-driven mesh refinement (basin AMR)	4
Lever T6 — Two-mesh projection: sidecar → fine background mesh → main mesh	4
4. Tool-to-failure mapping	4
5. Recommended phased plan	5
Phase F-1 — Quantify near-fault error precisely	5
Phase F-2 — DG $L^2(0)$ projection (Lever T1)	5
Phase F-3 — Per-side fault projection (Lever T3)	5
Phase F-4 — Sidecar-driven background field for gmsh (Lever T5)	6
Phase F-5 — Convergence study + ship	6
6. Recommended sequence of action	6
7. Existing-codebase compliance notes	7
8. Open questions for the user	7

Fault-Zone Projection-Quality Plan (v1)

Date: 2026-05-09 **Status:** Survey + phased plan (read before any implementation) **Triggering observation:**

Comparison of data_projected/preview/projected_velocity_500m against the sidecar at $z = -1$ km shows the **near-fault** mesh-projected field carries spurious 1–2 km wide low-Vs streaks tracking the cut fault trace (NW–SE band through the LA basin / SAF / Salton trough). Diff RMS = 0.466 km/s, max-abs = 2.57 km/s, with the worst residuals concentrated in tets immediately adjacent to the fault. Refining from 2000 m → 500 m mesh did **not** reduce the diff (RMS unchanged at 0.469 → 0.466 km/s).

The projection-quality v1 plan (miniapps/seas/document/features_dev/projection_quality_plan_v1.md) already covers the general source/projection/mesh decomposition. This document zooms in on the **fault-zone-specific** failure mode and catalogues the exact tools the existing seas codebase already exposes that we can compose to fix it.

1. Why the fault-zone region looks worst (precise diagnosis)

Three compounding effects, all rooted in the **H1-P1 / linear-in-tet** projection currently emitted by seas_project_velocity_to_mesh.

1.A — Fault-trace coincides with basin edges

The cut fault (NW-cut SAF strand) traverses **exactly** the regions where the sidecar's Vs field has its sharpest lateral gradients: LA basin, San Bernardino transition, San Andreas trough, Salton. At $z = -1$ km the Vs jumps from 0.5 km/s (basin) to 3 km/s (basement) over a few km, and the fault line follows that boundary.

1.B — Fault refinement creates a 2-D band of fine tets around a 1-D feature

safs_fault_box_nwcut.geo uses a `Field[Distance]` sized at `LC_NEAR = 1500 m` within `DIST_INNER = 3 km` of the fault, decaying to `LC_FAR = 10 km` beyond `DIST_OUTER = 40 km`. This produces a fine **3-D corridor** of ~ 1.5 km tets spanning ± 3 km around the fault surface. Inside this corridor:

- Vertices ON the fault are sampled by `ProjectCoefficient` at the fault location (often deep in the basin $\rightarrow$ very low Vs).
- Vertices a few km off the fault land in basement (high Vs).
- Linear-in-tet interpolation between them produces a **gradient smear** of low values diffusing 1–2 km out of the fault, visible in the middle panel as streaks.

This is geometrically asymmetric: the fault is a 2-D surface, but the finer tet density along it imprints a band that's wider than the sidecar's actual Vs gradient, painting a sub-km-thin geological boundary as a 1-2 km thick low-Vs layer.

1.C — H1-P1 forces continuity across the fault

Continuity of the FE basis means a single Vs value is “shared” across both $\pm$ sides of the fault face at any vertex on the fault interface. But the wave operator (`dynamic/wave_operator.hpp:608` — `L2_FECollection`, `GaussLobatto`) is **discontinuous Galerkin**:

- Each tet has its OWN per-element DOFs.
- The fault itself is treated as a discontinuity surface with separate $\pm$ states (`FaultBasis`, `FaultFace-Flux::Evaluate`).

Projecting material onto **H1-P1** is therefore **doubly inconsistent**: (1) it adds artificial continuity the simulation doesn't enforce, and (2) it cannot represent the real material discontinuity that may exist across the fault (e.g., basin vs. basement on opposite sides of the SAF — known to occur along SAFOD / Cajon).

2. Tools the codebase ALREADY provides

Every tool below is implemented in the seas miniapp; using them requires assembly, not new infrastructure.

Tool	Where (file:line)	What it gives us
<code>mfem::L2_FECollection</code> (<code>dynamic/wave_operator.inl:36</code> <code>dim, Gauss-</code> <code>Lobatto</code>)		Per-element discontinuous polynomial basis with the same nodal collocation as the wave operator's state . $p = 0 \rightarrow$ element-wise constant; $p = 1 \rightarrow$ linear within each tet, jumps at faces.
<code>mfem::DG_FECollection</code> (<code>dynamic/elasticity_operator_setup.inl:16</code> <code>dim, Gauss-</code> <code>Lobatto</code>) <code>FaultBasis +</code> <code>fault/fault_basis.hpp:62</code> , <code>fault_interior_dynamics/wave_operator.hpp:154</code> <code>/</code> <code>fault_shared_faces_</code> <code>BoundaryConfiguration</code> <code>dynamic/wave_operator.inl:171, 206, 287</code>		Equivalent to L2 with the same Gauss-Lobatto node distribution. Used by the elasticity domain operator already. Already enumerates fault faces as interior with <code>elem1 / elem2</code> (the $\pm$ sides). Lets us assign material per-side.
<code>mfem::ParGridFunctionCoefficient</code> (any FES)	<code>fault/fault_coefficient.hpp</code>	Tags the fault face attribute. Lets us filter 'fault-adjacent' tets. Already used; works with both H1 and L2.

Tool	Where (file:line)	What it gives us
mfem::QuadratureSpace (per-QP fields) mfem::FindPoints + QuadratureSpace	MFEM core 155-156 (linked)	Material at quadrature points directly — no FE projection at all.
mfem::Mesh::Refine AMR Field[Threshold] + BackgroundField	MFEM core 155-156 (linked)	High-quality scattered-point evaluation primitive; useful for non-conforming or volume-averaging projection. Refine specific elements based on a marker (e.g., Vs-gradient or basin-mask). Sidecar-driven mesh density at gen time.

The **two-state-per-fault-face** infrastructure (elem1 / elem2 plumbing throughout `fault_face_flux.cpp`, the `DOFData::Zp_plus / Zp_minus / Zs_plus / Zs_minus / eta_p / eta_s` in `fault/fault_basis.hpp:39` and downstream) means **per-side heterogeneous impedances are already a first-class concept in the runtime**; only the projection feeding it currently treats both sides as a single H1-continuous field.

3. Design space

Five orthogonal levers; pick a combination by physics target + compute budget.

Lever T1 — Switch material to $DG L^2(0)$ (per-element constant)

Each tet gets one (λ, μ, ρ) triple, computed from a centroid sample of the sidecar (or a volume average). Discontinuities at every face are **allowed** by construction; the fault is a special case of the generic face discontinuity.

- **Eliminates** §1.C entirely.
- **Eliminates** §1.B's band-smearing entirely (each tet stands alone; no interpolation across vertices).
- **Implementation:** swap `H1_FECollection(1, dim) → L2_FECollection(0, dim, GaussLobatto)` in `seas_project_velocity_to_mesh`; replace `ProjectCoefficient` (which point-samples at vertices) with **centroid sampling** OR a cubature-based volume average.
- **Cost:** low. Wave operator already consumes element-wise constant material when `MaterialField::Mode::Constant` (today's baseline); adding `Mode::DG_L2_GridFunction` is a parallel branch in `MaterialField::At` that indexes the per-element `GridFunction` by `(elem, 0)`.
- **Risk:** loses smooth in-tet variation (which we don't have anyway given source resolution).
- **Test:** for a constant-material baseline, $DG-0$ value should exactly equal the scalar; regression-cordon T-5-4..T-5-6 from the parent v2 plan still apply.

Lever T2 — $DG L^2(p \geq 1)$: keep within-tet curvature, allow face jumps

Same as T1 but with quadratic/cubic basis within each tet.

- Recovers **smooth** in-tet variation where source has it.
- Still allows face jumps (so basin/fault discontinuities are intact).
- **Implementation:** identical to T1 with `L2_FECollection(p, ...)`.
- **Cost:** higher — $p = 1$ quadruples per-element DOFs; $p = 2$ tenx. Wave operator must support order > 1 if reading from the same FES.
- **Recommendation:** start with T1 ($DG-0$). Move to T2 only if T1's basement smoothness is insufficient.

Lever T3 — Per-side fault-coupled projection

Without changing the FES, evaluate the sidecar **separately** for the elem1 and elem2 sides of every fault face: each tet that touches the fault is sampled at a centroid that has been **nudged into its own side** (e.g., centroid + $\varepsilon \times$ outward-normal), so a basin-side tet gets the basin sample and a basement-side tet gets the basement sample, no smearing.

- **Implementation:** in `field_coefficient.cpp::ProjectVelocity`, loop over `wave_op.GetFaultInteriorFaces` and for each face use the elem1/elem2 transformations to evaluate the coefficient at a per-side query point (`ip_centroid +  $\varepsilon \cdot$  normal_outward`).
- **Cost:** low — a few tens of LOC; reuses existing fault bookkeeping in `wave_operator.hpp:154`.
- **Constraint:** only meaningful with DG basis (T1/T2). Pointless on H1 because the H1 DOF on the fault is shared.
- **Pairs naturally with T1.**

Lever T4 — Volume averaging of the source field at projection time

For each sample location (centroid, vertex, or QP), evaluate the sidecar at **K sub-grid points** within a small ball (radius $\sim$ tet edge / 4) and average.

- Damps the fault-trace artifact at H1 layer too (smooths the spike the fault DOF would otherwise see).
- Preserves the integral mean of the source over each tet.
- **Implementation:** modify `FieldCoefficient::Eval` to do K-fold sampling and average; K = 8 (corner-of-cube) is enough for a noticeable improvement.
- **Cost:** trivial; K-fold extra interpolations $\times O(N_{\text{DOF}})$.
- **Risk:** smoothing also blurs basement away from the fault, so the source's own gradients suffer.

Lever T5 — Sidecar-driven mesh refinement (basin AMR)

Rather than fix the projection on the existing mesh, **refine the mesh** where the sidecar's Vs gradient is large, especially near basin-fault intersections.

- Most physically correct, biggest gain at convergence.
- **Implementation:** offline: sample $\|\nabla V_s\|$ on the sidecar grid, emit a gmsh `Field[PostView]` background-size field; the gmsh mesher then auto-tightens tets where the gradient is high.
- **Cost:** non-trivial offline pass; 2 $\times$ — 5 $\times$ larger meshes if the refinement is generous.
- **Pairs naturally with T1 (more elements + per-element constant = best of both worlds).**

Lever T6 — Two-mesh projection: sidecar $\rightarrow$ fine background mesh $\rightarrow$ main mesh

Project sidecar onto a **uniform fine cubical background mesh** at sub-source resolution (e.g., 250 m), then project that field onto the simulation mesh via L^2 -projection. The intermediate fine grid acts as a smoothing buffer.

- Probably overkill; same effect as T4 with explicit machinery.
- **Cost:** high; needs a second mesh + ParaView-side pipeline.
- **Recommendation:** skip unless T1 + T3 + T5 prove insufficient.

4. Tool-to-failure mapping

	§1.A basin/fault coincidence	§1.B fault-band smear	§1.C cross-fault continuity
T1 DG-0	(matters less; per-elem)	✓✓✓	✓✓✓
T2 DG-p $\geq$ 1	(depends on cell)	✓✓	✓✓✓

	§1.A basin/fault coincidence	§1.B fault-band smear	§1.C cross-fault continuity
T3 per-side fault projection	—	✓✓ (around fault)	✓✓✓
T4 volume averaging	—	✓ (smooths band)	—
T5 basin AMR	✓✓✓ (resolves edge)	(✓ side-effect)	—
T6 two-mesh	—	✓	—

The dominant fault-zone failure (visible as streaks in your image) is **§1.B + §1.C**. Both are eliminated by **T1 (DG-0) + T3 (per-side projection)**, and that combination is also what the wave operator already wants because it's already DG.

5. Recommended phased plan

Phase F-1 — Quantify near-fault error precisely

Goal: turn the visual diff plot into a number we can drive down.

Files: - New: safs/<project>/code_preprocess/projection_quality/fault_zone_metric.py

Requirements: 1. Read mesh-projected *.vtu and the sidecar .h5. 2. Identify mesh elements/DOFs **within R_fault metres of the fault surface** (use the cut STL: data_cutnwfault/SAFS-..._nwcut.stl → scipy.spatial.cKDTree on its triangle centroids → distance from each mesh vertex/centroid). 3. Report L^2 and L^∞ residuals against the sidecar **stratified by R_fault band**: [0–500 m], [500–1500 m], [1500–3000 m], [> 3000 m]. 4. Output JSON + a marker VTU showing the per-element residual.

Acceptance: - The [0–500 m] band's L^2 error is reported separately from the bulk; we should see it $\geq 5\times$ the [> 3000 m] band on the current 500 m mesh.

Phase F-2 — DG $L^2(0)$ projection (Lever T1)

Goal: material projected onto a `L2_FECollection(0, dim)` GF; one value per tet; the wave operator's `MaterialField` consumes it.

Files: - Modified: miniapps/seas/io/field_coefficient.{hpp,cpp} — add `ProjectVelocityDG(sidecar_path, fes_l2_const)` and a per-element centroid-sampling path. - Modified: miniapps/seas/dynamic/heterogeneous_material.hpp — add `Mode::DG_L2_GridFunction` whose `At(elem, dof)` returns the same value for any dof (since $L^2(0)$ has 1 DOF per element). - Modified: miniapps/seas/drivers/project_velocity_to_mesh.cpp — `--basis dg0|h1p1` flag; default `dg0`.

Requirements: - `ProjectCoefficient` on an `L2_FECollection(0, ...)` `GridFunction` must produce, for each tet, a value equal to the centroid sample of the underlying coefficient (the trilinear sidecar interp). - For a constant-material baseline ($V_s \equiv 2500$, $V_p \equiv 4000$, $\rho \equiv 2700$), DG-0 GF must equal the scalar to within float precision.

Acceptance (against F-1 metric): - Near-fault [0–500 m] band L^2 error drops by **at least 3x** vs. H1-P1 baseline. - Bulk error (≥ 3 km from fault) **does not** regress.

Cost: small. Only reuses existing infrastructure.

Phase F-3 — Per-side fault projection (Lever T3)

Goal: every tet that touches the fault is sampled in its own half-space; the projection respects the fault as a discontinuity surface even where the sidecar happens to be smooth across it.

Files: - Modified: `miniapps/seas/io/field_coefficient.{hpp,cpp}` — new `ProjectVelocityDG_FaultAware(...)` that: 1. Calls `ProjectVelocityDG` first (T1 baseline). 2. Iterates over `wave_op.GetFaultInteriorFaces()` (faces tagged `fault_attr`). 3. For each face: get `(elem1, elem2)` via `ParMesh::GetFaceElementTransformations(face_idx)`. 4. For each side $s \in \{1, 2\}$: query the sidecar at $\text{centroid}(\text{elem}_s) + \varepsilon \cdot \mathbf{n}_{s_outward}$ where ε is e.g. $0.1 \times \text{LC_NEAR} / \cos(\text{angle})$, and overwrite that tet's DOF. 5. Use `FaultBasis::GetFaultQPs()` for the normal direction.

Requirements: - Near-fault tet whose centroid sits in basin gets the basin sample; the matching cross-fault tet gets the basement sample, even if a naive centroid would round into the wrong region. - Existing tests for `ProjectVelocity` still pass.

Acceptance (against F-1 metric): - Near-fault $[0-500 \text{ m}]$ band L^2 error drops by **at least 2x** vs. F-2 alone. - Demonstration: pick a fault-crossing transect through the LA basin and plot Vs on each side of the fault; the discontinuity must align with the fault face, not with the nearest tet vertex.

Cost: moderate. ~ 150 LOC; depends on `wave_op` being constructed before projection (or on extracting the fault-face list directly from the mesh in `field_coefficient.cpp`).

Phase F-4 — Sidecar-driven background field for gmsh (Lever T5)

Goal: mesh refinement that doesn't just refine near the fault but specifically near basin/basement boundaries (high $\|\nabla V_s\|$).

Files: - New: `safs/<project>/code_preprocess/data_projection/build_size_field.py` — sample $\|\nabla V_s\|$ on the sidecar grid, write a gmsh `.pos` (PostView) file with a target tet size that's small where the gradient is large. - Modified: `safs/<project>/code_meshing/safs_fault_box_nwcut.geo` — load the `.pos` as a `Field[PostView]`; combine with the existing fault distance field via `Field[Min]`. - Modified: `safs/<project>/code_meshing/run_nwcut_meshing.py` — optional `--size-field-pos PATH` flag.

Requirements: - The `.pos` file has tet-size values everywhere the mesh might land (i.e., over the velocity inscribed AABB plus padding). - gmsh produces a mesh whose tet size in basin-edge regions is $\leq \text{LC_NEAR} / 2$ (e.g., 750 m) without exceeding $2 \times$ the baseline total tet count.

Acceptance: - F-1 metric on the new mesh: $[0-500 \text{ m}]$ band L^2 down by $\geq 3 \times$ AGAIN beyond what F-2 + F-3 achieve.

Cost: moderate. Requires gmsh `.pos` generator + a re-mesh.

Phase F-5 — Convergence study + ship

Same as Q-5 from the parent plan, restricted to the fault zone. Tabulate (`mesh_h` $\times$ `basis` $\times$ `per-side proj` $\times$ `size-field`) against the F-1 fault-zone metric. Ship the recommended combination.

6. Recommended sequence of action

Order	Phase	Why this order
1	F-1: quantify	We need a number, not a picture, before changing anything.
2	F-2: DG-0 projection	Largest single fix to §1.C; aligns material with wave-op basis; cheapest implementation.
3	F-3: per-side fault sampling	Pairs naturally with DG-0; eliminates the fault-band smear for free once the FES is L^2 .
4	(assess)	If F-2 + F-3 hit acceptance criterion $\rightarrow$ ship. Otherwise $\rightarrow$ F-4.
5	F-4: sidecar-driven AMR	Final reduction if F-2 + F-3 are insufficient.

Order	Phase	Why this order
6	F-5: convergence + ship	Doc + sweet-spot table.

Do NOT go straight to F-4 or T6. The first observable fix that the user-visible diff plot will respond to is **F-2 + F-3**, because they directly attack the failure modes documented in §1.B and §1.C — which are the actual reasons the streaks appear.

7. Existing-codebase compliance notes

- **CLAUDE.md C2 invariant:** no edits to bp5/, bp1/, bp2/, domain/, friction/dieterich_ruina.hpp, solver/, or fault/fault_basis.hpp. All proposed work fits in io/, dynamic/heterogeneous_material.hpp, drivers/, and the offline preprocess directory.
- **DG-mode MaterialField** is the natural extension of `Mode::Constant: At(elem, dof) → (λe, μe, ρe)` ignores dof for L2(0). Same accessor shape; existing TPV102/TPV104/TPV205/BP5 drivers continue to use `Mode::Constant` (byte-identical).
- **One-time-load contract** (parent C-6) is preserved across all proposed levers: the projection happens once at driver init.

8. Open questions for the user

1. What is the **near-fault accuracy target**? $\leq 5\% L^2$ inside [0–500 m] band, or tighter?
2. Do you expect **pre-existing fault-perpendicular Vs jumps** in the model (basin vs basement on opposite sides of the SAF)? If yes, T3 is essential. If no, T1 alone may suffice.
3. Is **F-4 (mesh refinement)** politically acceptable — i.e., are you OK with a 2-3× larger mesh (and the corresponding wave-op compute cost)? If not, F-2 + F-3 is the ceiling.

Send answers and I'll write the F-2 implementation plan in file-level detail.